

Foundation for Advancement of Education and Research

G5, Swiss Complex, 33, Race Course Road, Bangalore - 560001

E-mail: office@faer.ac.in, Website: www.faer.ac.in

FAER Scholar Award Program – A Multidisciplinary

Engineering Innovation Challenge: 2026-2027

Proposal Format

- | | |
|--|---|
| 1. Title of the Project | : Phoenix: A Self-Healing Recovery Fabric for Fault-Tolerant AI Training Infrastructure |
| 2. College Name | : Kamaraj College of Engineering and Technology |
| 3. City and State | : Virudhunagar , Tamil Nadu |
| 4. Names of the Students | : Padma Munish Dhanajeyan.T, Dhawanth Raj.N |
| 5. Degree registered | : BE |
| 6. Branch | : CSE |
| 7. Year | : III year |
| 8. Address (residential) | : No 1/2 Jawahar sangam street, SBK Boys School road, Aruppukottai-626101 |
| 9. E-mail address | : 24ucs027@kamarajengg.edu.in |
| 10. Phone Number | : 9843057938 |
| 11. Name and Address of the College
(including pin code) | : Kamaraj College of Engineering and Technology
S.P.G. Chidambara Nadar – C. Nagammal Campus,
S.P.G.C. Nagar, K. Vellakulam – 625 701 |
| 12. Web site of the College | : kamarajengg.edu.in |
| 13. Phone No | : (+91) 4549-278791 |
| 14. Name of the Supervisor /
Project Guide with E-mail address
and mobile Number | : Dr. G. Uma Maheswari
umamaheswaricse@kamarajengg.edu.in |
| 15. Whether the students belong to
SC/ST | : No |
| 16. Title of the Project | : Phoenix: A Self-Healing Recovery Fabric for Fault-Tolerant AI Training Infrastructure |
| 17. Area | : Artificial Intelligence and Systems Infrastructure |

18. Relevance to the area of the contest :

Modern Artificial Intelligence relies on training large language models and multi-billion parameter foundation architectures across massive distributed clusters containing thousands of GPUs. At this scale, hardware failure is not an occasional anomaly—it is a continuous, statistical certainty. In published empirical data from Meta's Llama 3 training run across 16,384 GPUs, the cluster suffered 419 unexpected interruptions within 54 days, averaging an interruption every three hours. At hyperscale deployments exceeding 100,000 accelerators, node failures are projected to occur roughly every 14 minutes.

Because distributed AI training workloads utilize tightly synchronized communication collectives (such as NCCL rings where every GPU must exchange state at every step), a failure in even a single worker node halts the entire training job. The current industry-standard recovery process requires detecting the failed worker, rescheduling a replacement node, reading a multi-hundred-gigabyte checkpoint from shared persistent storage, reinitializing the software environment, and rebuilding all communication groups. This standard cold-restart sequence consumes between 4 to 20 minutes of complete cluster downtime per failure event. Industry studies indicate that checkpoint and recovery overhead consumes 12% to 43% of total training time, generating hundreds of millions of dollars in idle compute waste on frontier-scale training campaigns.

The relevance of this project lies in bridging Artificial Intelligence and Computer Systems Infrastructure. While the workload being safeguarded is large-scale AI model training, the core engineering solution is implemented using low-level systems programming, hypervisor virtualization (KVM/QEMU), and server-side failover orchestration. By engineering an open-source, resilient recovery framework, Phoenix transforms fragile AI training setups into a self-healing fabric, directly aligning with FAER's theme of technology-enabled engineering innovation.

19. Type of Project : Product (Systems Software Framework)

20. Objective of your project :

- To design and deploy a fast-recovery failover framework for distributed AI training workloads that reduces cluster-wide recovery idle time from minutes to tens of seconds.
- To implement live, incremental memory-state checkpointing between an active compute node and a pre-warmed standby node using KVM/QEMU hypervisor virtualization.
- To build an intelligent, lightweight server-side failure detection controller that reliably distinguishes transient network slowness from catastrophic node failure, eliminating false failovers.
- To automate standby node promotion and socket redirection so training resumes with minimal state loss.

- To establish a rigorous, repeatable benchmarking pipeline providing quantitative timestamped recovery comparisons between Phoenix and conventional cold-restart baselines.
- To keep the entire framework 100% open-source, hardware-modest, and fully reproducible using a standard laptop and college lab servers.

21. Brief description :

Phoenix is an open-source, systems-level fault-tolerance framework designed to eliminate idle-time waste in distributed AI training infrastructure. In existing deep learning cluster architectures, node failures are handled reactively through a cold-restart paradigm: when an instance fails, the cluster scheduler terminates the container, provisions a replacement machine from scratch, loads the most recent monolithic checkpoint from disk, re-establishes GPU cluster network collectives, and resumes execution. This approach incurs severe latency overheads, stranding thousands of healthy GPUs while waiting for the single node to recover.

Instead of treating recovery as a cold re-initialization, Phoenix establishes an active-to-standby synchronization pipeline utilizing low-level hypervisor virtualization (KVM/QEMU). A pre-warmed standby virtual node is maintained alongside the active compute worker. Rather than transferring massive full-memory snapshots across the network at periodic intervals, Phoenix employs live incremental dirty-page tracking. During continuous operation, only the modified memory pages (forward incremental checkpointing) are asynchronously streamed to the standby machine, keeping synchronization overhead negligible.

A server-side health-monitoring daemon tracks heartbeats, memory pressure, and socket responsiveness of the active training node. The controller is calibrated with sliding-window telemetry to distinguish transient networking latency or garbage collection stalls from genuine node hardware crashes. Upon confirming an unrecoverable failure, the failover orchestrator instantly executes a zero-downtime promotion of the standby node. Because the standby already possesses a near-identical live memory image, it resumes compute execution in ~20 seconds—bypassing disk checkpoint loading and deep re-initialization.

Crucially, Phoenix focuses strictly on the computer systems and infrastructure layer beneath the AI models: hypervisor virtualization, memory page dirty-logging, health monitoring, and server-side socket failover. The entire software framework is architected to run on standard Linux environments and commodity hardware (such as personal development laptops and university lab servers) using entirely open-source tooling, proving that resilient AI infrastructure can be accessible, reproducible, and cost-effective.

22. Implementation Plan from

December 2026 to May 30, 2027 :

- **December 2026:** Environment configuration & virtualization setup. Install and tune KVM/QEMU on local development laptop and lab server; establish virtual network bridge and configure two virtual nodes (active and pre-warmed standby).

- **January 2027:** Implementation of live incremental checkpointing. Configure QEMU dirty-bitmap tracking and memory page streaming between active and standby nodes; optimize memory synchronization intervals.
- **February 2027:** Development of health-check controller. Build lightweight telemetry daemon; calibrate timeout and sliding-window heuristic algorithms to reliably differentiate transient latency from fatal node crash.
- **March 2027:** Failover logic & automated promotion. Implement fast standby promotion scripts, virtual network IP/socket rerouting, and end-to-end timestamped instrumentation across all failover phases.
- **April 2027:** Benchmarking & failure-injection testing. Execute controlled, automated failure injections comparing Phoenix against conventional cold-restart baselines; record quantitative recovery metrics (mean, variance, memory overhead).
- **May 2027:** Experimental analysis, documentation & final reporting. Compile comprehensive performance graphs, prepare open-source code repository, and draft the final project report for FAER submission.

This phased plan ensures that by May 2027, a fully functional prototype is available, tested, and ready for demonstration.

23. Existing Approaches :

Existing responses to AI training failures predominantly depend on periodic checkpointing to network-attached storage (NFS, Ceph, or S3). In empirical studies of production clusters (such as Meta's technical report on Llama 3 training), when a GPU node experiences an ECC memory fault, kernel panic, or NVLink drop, the entire synchronous training job is paused. The cluster scheduler (Slurm or Kubernetes) terminates the affected container, provisions a spare node, pulls the latest saved model weights from disk storage, and rebuilds the NCCL communication ring before resuming execution. This workflow results in substantial cluster idle latency.

To mitigate this, commercial cloud providers have recently engineered proprietary solutions. For example, Crusoe Cloud developed 'AutoClusters' to rapidly replace failed GPU instances, and commercial startups like Clockwork.io offer closed-source cluster resilience tools. However, these systems are locked behind proprietary vendor platforms, require specialized hardware, or are inaccessible to academic institutions, research labs, and independent developers. On the orchestration side, standard Kubernetes controllers were architected for stateless web microservices that restart cleanly, lacking native awareness of tightly coupled, stateful AI workloads.

24. What is new in the project :

Phoenix introduces a novel application of hypervisor-level incremental live memory synchronization directly tailored to stateful AI training resilience. Rather than relying on static disk snapshots or proprietary hardware appliances, Phoenix adapts open-source QEMU/KVM

dirty-page tracking to maintain a continuously synchronized, pre-warmed standby node. The project delivers an open, modular server-side architecture featuring:

- Fine-grained incremental memory replication that streams only modified memory pages, dramatically cutting network synchronization overhead.
- A sliding-window failure detection algorithm that prevents catastrophic false failovers caused by transient network jitter.
- A hardware-agnostic design running on standard commodity laptops and open-source Linux virtualization, making state-of-the-art fault tolerance verifiable and reproducible without multi-million-dollar GPU clusters.

25. How is this proposal different from existing approaches based on :

Originality :

While memory migration has been explored for web servers, Phoenix pioneers the adaptation of hypervisor dirty-page streaming to distributed AI training nodes. It transitions cluster recovery from an uncoordinated 'stop-everything-and-reload' model to a continuous, hot-standby failover architecture, publishing an open reference implementation accessible to the academic community.

Performance :

Conventional checkpoint-and-restore sequences stall clusters for 4 to 20 minutes while gigabytes of weights and optimizer states are pulled from remote storage. By pre-warming the standby instance with streaming memory delta pages, Phoenix aims to complete failover and resume execution within 20 to 30 seconds, yielding an order-of-magnitude reduction in recovery time.

Usability, affordability :

Unlike commercial enterprise tools that demand specialized hardware, vendor-locked clouds, or proprietary licenses, Phoenix is built entirely upon native Linux virtualization tools (KVM, QEMU, Python). It is fully demonstrable and testable on a standard student laptop and local college lab servers, offering zero licensing fees and immediate reproducibility.

26. References (Literature Survey) :

- **Meta AI, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024** – Technical report documenting large-scale distributed training infrastructure across 16,384 GPUs, recording 419 unexpected failure interruptions over 54 days and detailing the severe cluster idle overhead incurred by standard checkpoint-and-resume procedures. <https://arxiv.org/abs/2407.21783>
- **QEMU / KVM Project Documentation, "Live Migration and Incremental Memory Dirty-Page Tracking," 2024** – Official technical documentation detailing hypervisor-level memory dirty-bitmap tracking, continuous incremental state transfer, and live guest failover mechanisms <https://www.qemu.org/docs/master/devel/migration.html>

27. Approach to solve a problem :

Modelling

- Formulate a distributed failure state machine classifying node conditions into Healthy, Degraded/Transient, and Unrecoverable Failure.
- Model memory dirtiness rate during compute workloads to establish optimal dirty-page synchronization frequency balancing network overhead against recovery freshness.

Construction / Development

- Configure a virtual cluster environment using KVM/QEMU with an active compute guest and a pre-warmed standby guest.
- Implement the memory synchronization pipeline leveraging QEMU dirty-bitmap tracking over a local virtual network bridge.
- Develop the server-side failover controller in Python to monitor instance health, execute standby promotion, and dynamically remap socket endpoints.

Experiments / Programming

- Develop an automated failure-injection script that simulates sudden power-off, kernel panic, and network isolation on the active node under active load.
- Implement a conventional cold-restart baseline runner (simulating process restart and checkpoint disk loading) for side-by-side evaluation.
- Embed microsecond-resolution timestamping at detection, promotion, and compute resumption milestones.

Testing

- Unit Testing: Validate socket heartbeat health detection, dirty-page capture routines, and promotion hooks independently.
- Integration Testing: Validate seamless end-to-end failover from active crash to standby resumption under synthetic compute workloads.
- Stress & Sensitivity Testing: Evaluate performance under varying memory mutation rates and simulated network degradation.

Results

- Empirical reduction in failover recovery time from ~4 minutes (baseline cold restart) to ~20-30 seconds using Phoenix.
- Quantified memory synchronization overhead demonstrating <5% performance degradation on the active worker during continuous operation.
- Zero false-positive failovers under simulated transient network delays.

System integration issues :

- Managing resource contention between dirty-page memory streaming and active computational throughput on modest test hardware.
- Tuning heartbeat timeout windows to avoid premature failovers during brief CPU/memory saturation spikes.

28. Support infrastructure required :

Hardware Requirements

- Personal Development Laptop (Intel/AMD x86_64 architecture with hardware virtualization support VT-x/AMD-V) — already available with the team.
- College Department Computer Laboratory Workstation / Server for multi-VM scaling verification — accessible in the college.

Software Requirements

- Operating System: Linux (Ubuntu 22.04 / 24.04 LTS) — Free & Open Source.
- Hypervisor & Virtualization: QEMU / KVM, libvirt — Free & Open Source.
- Programming & Controller Languages: Python 3.10+, Bash scripting — Free & Open Source.
- Version Control: Git and GitHub for project tracking and collaboration.

Networking Requirements

- Local virtual bridge networking (virtbr0 / TAP interfaces) for inter-VM synchronization without requiring external network bandwidth.

Human Resources

- Student Developer 1 (Padma Munish Dhanajeyan.T): Virtualization layer, QEMU dirty-bitmap tracking, and live memory streaming pipeline.
- Student Developer 2 (Dhawanth Raj.N): Failure detection controller, sliding-window heuristics, and experimental benchmarking.
- Project Supervisor (Dr. G. Uma Maheswari): Architectural guidance, systems research review, and project milestone validation.

Additional Support

- Department laboratory access and open-source Linux kernel & QEMU development documentation.

29. Reasons why this proposal should be considered for selection :

The problem addressed by Phoenix is one of the most critical and economically consequential operational bottlenecks in the global artificial intelligence ecosystem. As demonstrated by published data from frontier AI organizations like Meta, hardware failure in massive GPU clusters is statistically relentless, leading to hundreds of millions of dollars in idle computational waste. While commercial hyperscalers develop proprietary, closed solutions, no open-source, standardized infrastructure framework exists for academic labs, sovereign AI efforts, or resource-constrained research teams.

This proposal does not make unsubstantiated claims to re-engineer multi-thousand GPU superclusters; rather, it delivers an honest, rigorous, and scientifically defensible systems contribution: demonstrating that hypervisor-level incremental memory checkpointing and intelligent server-side failover can slash recovery times from minutes to seconds on accessible, open-source software.

Furthermore, this project embodies the highest ideals of the FAER Scholar Challenge. It moves beyond generic, superficial machine learning applications to deliver fundamental computer systems and infrastructure engineering. It requires zero budget, utilizes existing student hardware and freely available open-source tools, presents an achievable 5-to-6-month experimental plan, and produces verifiable quantitative benchmark data. Selecting Phoenix represents an investment in high-impact, foundational systems engineering that directly aligns with national priorities in computing resilience and digital infrastructure.

30. Budget Estimate :

ITEM	Number of Quantities	COST (₹)
Personal Development Laptop (Intel/AMD Virtualization)	1 (Self-owned)	Free open Source
Linux OS & QEMU/KVM Virtualization Stack	Free Open Source	
Python, Libvirt & Testing Scripts	Free Open Source	
Total	—	Not Applicable

DECLARATION

We will take up this project if selected, for FAER Scholar Award Program: A Multidisciplinary Engineering Innovation Challenge: 2026-2027 only and will not submit this to any other contest. We will comply with FAER's requests on submission of action plans, summary, monthly progress reports. We will regularly discuss with mentors and follow their advice. Mentors are there to help us.

Signature of Students

1)Padma Munish Dhanajeyan
2)Dhawanth Raj.N

Signature of Project Guide

1)Dr.G.Uma Maheswari

Date:

Signature of Principal

DECLARATION / UNDERTAKING

This is to certify that Mr. Padma Munish Dhanajeyan.T(24UCS027) and Mr. Dhawanth Raj.N (24UCS023) are Bonafide students of III year in the B.E. Computer Science and Engineering degree program of our college. If the proposal is selected by FAER, we will provide the requisite laboratory / computer support in our college.

Date:

Place: Virudhunagar

Signature of Principal

College Name: Kamaraj College of
Engineering and Technology